SAPIENTIA
ERDÉLYI MAGYAR
TUDOMÁNYEGYETEM
Marosvásárhelyi Kar

Virtual Mouse

Makkai Nándor
Sapientia EMTE
Szoftverfejlesztés I. év
Szoftver rendszerek tervezése

Marosvásárhely

2022

1. Bevezetés

Az ember és számítógép közötti interakciók (HCI) megvalósulásáért számos eszköz jött létre. Ilyen például az egér, amely segítségével a felhasználó mozgatni tudja a kurzort és különböző akciókat hajthat végre vele. Az évek során megjelentek a kábel nélküli, Bluetooth-os egerek, ezek működése még mindig függ egyéb eszközöktől, például az elemtől, akkumulátortól, amely az áramot szolgáltatja, vagy a dongle-től, amelyet a számítógépbe kell helyezni az egér működéséhez.

2. Célkitűzések

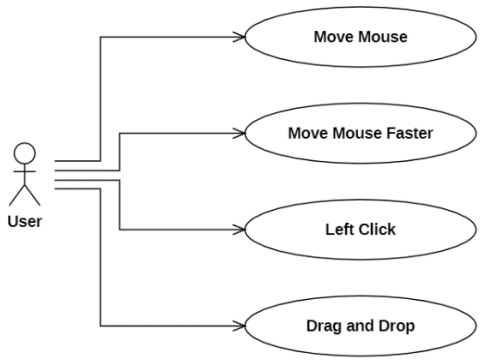
A projektem célja ennek a függőségnek a feloldása és a fizikai egér helyettesítése volt egy virtuális egér által. Ez a rendszer lehetővé teszi az egérkurzor mozgatását és az egér más funkcióinak elvégzését csupán a kézmozdulatok használatával, felismerésével. Ezzel a megoldással a felhasználónak csak egy számítógéphez csatlakoztatott kamerára van szüksége.

3. Használt technológiák

A rendszer fejlesztése Python programozási nyelven történt Visual Studio Code fejlesztői környezteben. A képfeldolgozás megvalósításához az OpenCV könyvtár csomagot használtam, a kéz felismerését és a kézmozgás nyomon követését a MediaPipe segítségével oldottam meg. A kézmozdulatokhoz tartozó egérfunkciókat a PyAutoGUI python csomaggal valósítottam meg.

4. Felhasználói követelmények

A rendszer használatához a felhasználónak rendelkeznie kell egy számítógéphez csatlakoztatott kamerára. Ezen kívül szükség van egy telepített Python interpreterre, amellyel futtatni lehet a programot. A program használatához a felhasználónak ismernie kell a kézmozdulatokat, amelyek kiváltják az egér funkciókat. A kézmozdulatok az egyes felemelt ujjakat jelentik és a rendszer a következőkre tud reagálni:



1. ábra Use Case diagram

- Mutató ujj felemelve: egérkurzor mozgatása
- Mutató és középső ujj felemelve: egérkurzor gyorsabb mozgatása
- Mutató és kisujj/Mutató, középső és kisujj felemelve: klikkelés (bal), közben nem mozgatható a kurzor
- Mutató, középső, gyűrűs és kisujj felemelve: bal egérgomb lenyomása, közben mozgatható a kurzor (drag)

5. Rendszerkövetelmények

5.1. Funkcionális követelmények

A rendszernek tudnia kell a számítógéphez csatlakoztatott kamera által szolgáltatott képen felismerni az emberi kézt, meghatározni, hogy mely ujjak vannak felemelve és ettől függően az adott egér funkcionalitást elvégezni. Az egér funkcionalitások a következők: kurzor mozgatása, klikkelés, drag-and-drop.

5.2. Nem funkcionális követelmények

A rendszer működéséhez szükséges egy kamera. A MediaPipe a kézfelismerést a CPU-n végzi, ezért a teljesítmény érdekében szükség van egy jó minőségű processzorra. A program futtatásához Python interpreterre van szükség, valamint a mediapipe, opencv és pyautogui Python csomagokra, a szükséges csomagok teljes listája megtalálható a requirements.txt fájlban. A fejlesztés Windows operációs rendszeren történt, más rendszereken nem volt tesztelve a működése, ezért ajánlott Windows alatt futtatni.

6. Fejlesztés menete

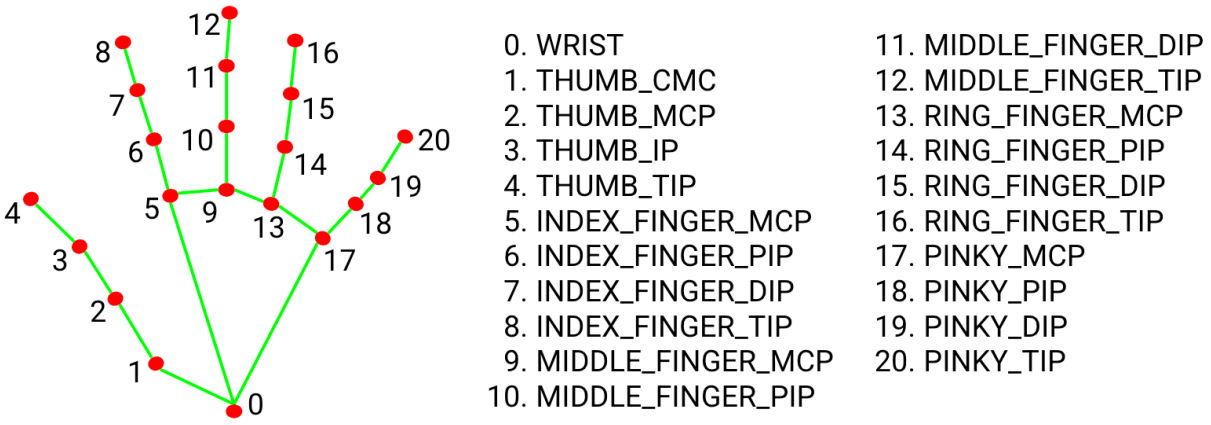
A fejlesztést azzal kezdtem, hogy létrehoztam egy GitHub repositoryt és egy kanban boardot a Trello alkalmazásban, ahol számon tudtam tartani a feladatokat. Megterveztem a kézmozdulatokat és a hozzájuk tartozó egér funkciókat. Ezt követően létrehoztam egy virtuális python környezetet a virtualenv Python csomag használatával és ezen az izolált környezeten belül fejlesztettem.

6.1. Kézfelismerés

A kézfelismerés megvalósításához a MediaPipe-ot használtam, amely gépi tanuláson alapuló megoldásokat nyújt, például a kézfelismerést. A kamera képét az Opencv segítségével kaptam meg és adtam át a MadiaPipe-nak feldolgozásra. Itt figyelni kellett arra, hogy az Opencv BGR színmegjelenítésű képet ad, a MediaPipe viszont RGB képekkel dolgozik, ezért át kell alakítani a képet.

A rendszer jelenleg egy kéz felismerésére van kialakítva. Készítettem egy *HandTracker* osztályt, amely a kézfelismerést valósítja meg. A konstruktorban létrehozza a szükséges MediaPipe objektumokat. A konstruktor négy paramétere által személyre szabhatjuk a kézfelismerőt:

- static_mode (default: False): a feldolgozás módját adja meg, ha False, akkor videó bemenetként tekint a képekre, ez azt jelenti, hogy az első képkockán felismeri a kezet és a következőkön pedig nyomon követi; ha True, akkor minden bemeneti képkockára elvégzi a kézfelismerést.
- max_hands (default: 2): a maximális felismerhető kezek száma.
- detection_con (default: 0.5): a felismerés konfidencia értéke.
- tracking_con (default: 0.5): a nyomon követés konfidencia értéke.



2. ábra Hand landmarks

Az osztálynak van további két metódusa. Az egyik a find_hands, amely a konkrét kézfelismerést valósítja meg. Bemenetként megkapja a képet, átalakítja RGB képpé és lefuttatja rajta a MediaPipe kézfelismerést. A MediaPipe Visszaadja a felismert kezek pontjainak pozícióit (lásd: 2. ábra). Ezek a pozíciók 0 és 1 közötti értékek, ezért a függvény átalakítja őket pixel koordinátákká, az eredményt pedig egy privát adattagban tárolja.

A másik metódus lehetővé teszi egy megadott kéz pozícióinak lekérdezését a privát adattagból. Ez a get_landmarks függvény, amely bemenetként a kéz indexét várja és ez alapján visszatéríti a kéz pontjainak koordinátáit.

6.2. Kézmozdulat felismerése

A kézfelismerés után meg kell határozni a kézmozdulatot, vagyis azt, hogy melyik ujjak vannak felemelve. Erre a célra létrehoztam egy HandGestureDetector osztályt, amely a kézfelismerés által szolgáltatott kéz pozíciókból meghatározza ezt. Az elgondolás az volt, hogy a rendszer akkor is meg kell tudja mondani, hogy egy ujj fel van-e emelve, amikor elfordítva vagy teljesen

lefele tartja a felhasználó a kezét. Ennek érdekében ez adott kéz pozíciók közötti távolságok kiszámításával történik. Mivel a hüvelykujj különbözik a többitől és nem találtam hatékony megoldást, ami megmondja, hogy fel van emelve az ujj vagy nincs, ezért csak a másik négy ujjat vettem figyelembe a kézmozdulatok detektálásánál. A detektálás során minden ujj esetében három távolságot számol:

- d1: a 0 és az ujj első pontja (5, 9, 13, 17) közötti távolság
- d2: a 0 és az ujj harmadik pontja (7, 11, 15, 19) közötti távolság
- d3: a 0 és az ujj negyedik pontja (8, 12, 16, 20) közötti távolság

Amennyiben a d3 nagyobb a d1-nél és a d2-nél, az ujj fel van emelve. A függvény a felemelt ujjak tekintetében meghatározza, hogy milyen műveletet kell végrehajtani:

- Mutató ujj felemelve: egérkurzor mozgatása
- Mutató és középső ujj felemelve: egérkurzor gyorsabb mozgatása
- Mutató és kisujj/Mutató, középső és kisujj felemelve: klikkelés (bal), közben nem mozgatható a kurzor
- Mutató, középső, gyűrűs és kisujj felemelve: bal egérgomb lenyomása, közben mozgatható a kurzor (drag)

A függvény kimenete az elvégzendő művelet és egy tömb az ujjak helyzetéről, vagyis arról, hogy fel van emelve vagy sem.

6.3. Egér funkcionalitások

Az egér funkcionalitások megvalósításáért a VirtualMouse osztály felel, amely a PyAutoGui Python csomagot használja. A PyAutoGui függvényei blokkolják a fő szálat, ezért az osztály a threading csomag segítségével egy háttérben futó szálon futtatja ezeket a függvényeket.

Az egér mozgatása relatívan történik a mutató ujj pozíciójához képest. A normál mozgatás során az előző pozíció és a mostani pozíció különbségének a kétszeresével mozdul el az egérkurzor, míg a gyorsabb mozgatásnál az ötszörösével.

A klikkelés a kisujj felemelésével történik, mindig csak egyszer, a következő klikkelés akkor következik be, amikor leengedjük a kisujjat és újra felemeljük. Eközben az egér nem mozgatható.

A drag-and-drop funkcionalitás akkor lép érvénybe, amikor mind a négy ujj fel van emelve (kivéve a hüvelykujjat), ekkor megtörténik a bal egérgomb lenyomása. Közben lehet mozgatni az egérkurzort és amikor leengedjük valamelyik ujjunkat, akkor véget ér a funkció, elengedi a bal egérgombot.

7. Összegzés és jövőbeli tervek

Összefoglalva, a fejlesztés során sikerült tapasztalatot szereznem, elmélyítenem a Python programozási tudásomat. Úgy gondolom, hogy a projekt sok szempontból tovább fejleszthető:

- több egér funkcionalitás implementálása
- kézmozdulatok újra tervezése, hogy két kézzel legyen használható a rendszer, ez megkönnyítené a több egér funkció implementálását
- testreszabhatóság a felhasználók által, pl. lehessen állítani az egér érzékenységét

Projekt GitHub linkje: https://github.com/MakkaiNandor/virtual-mouse

8. Bibliográfia

https://www.hindawi.com/journals/jhe/2021/8133076/

https://ieeexplore.ieee.org/abstract/document/8934612

https://ai.googleblog.com/2019/08/on-device-real-time-hand-tracking-with.html

https://google.github.io/mediapipe/solutions/hands.html